

Possible B-states from an Action

$$\text{RESULT}(b, a) = \{b_o \mid o \in \text{POSSIBLE-PERCEPTS}(\text{PREDICT}(b, a))\}$$

Includes all the belief states consistent with one possible percept after the action a is applied to the believe state b .

Possible B-states from an Action

$$\text{RESULT}(b, a) = \{b_o \mid o \in \text{POSSIBLE-PERCEPTS}(\text{PREDICT}(b, a))\}$$

Includes all the belief states consistent with one possible percept after the action a is applied to the believe state b .

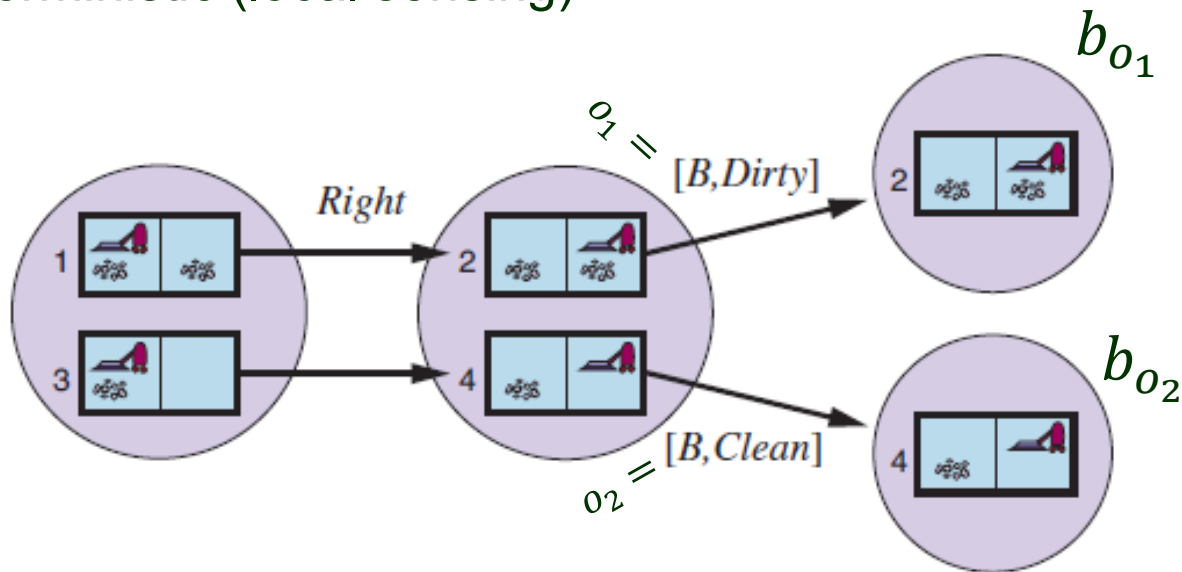
- Deterministic (local sensing)

Possible B-states from an Action

$$\text{RESULT}(b, a) = \{b_o \mid o \in \text{POSSIBLE-PERCEPTS}(\text{PREDICT}(b, a))\}$$

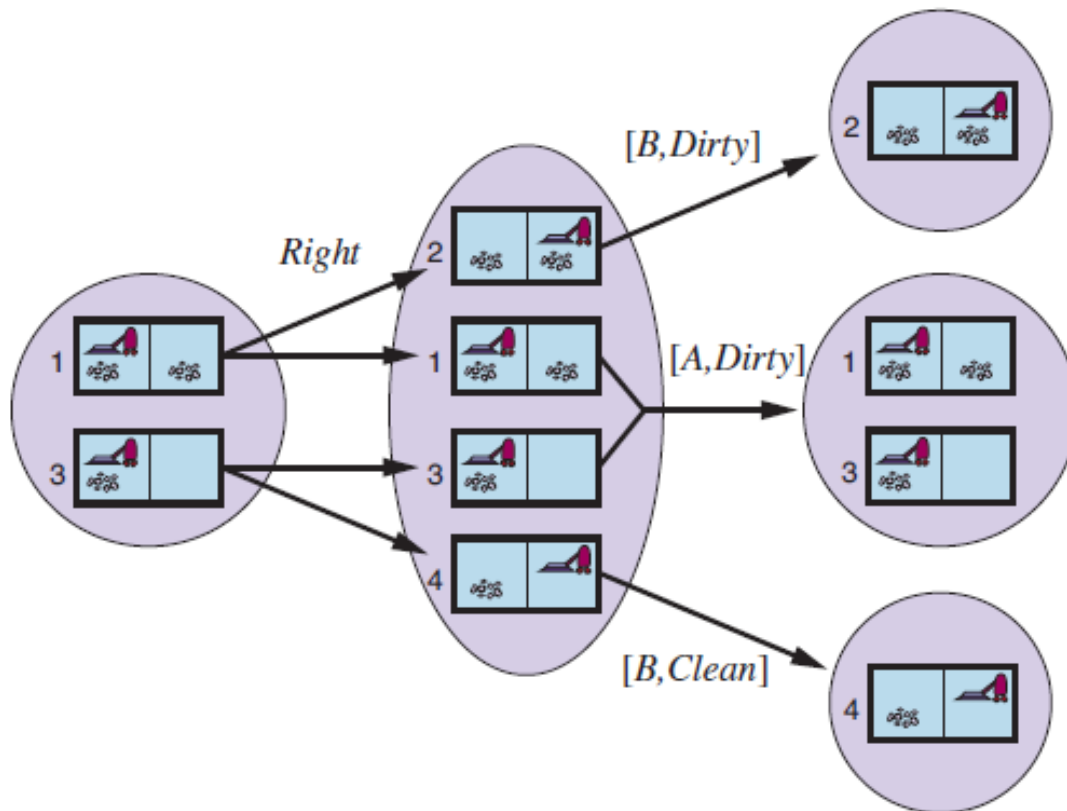
Includes all the belief states consistent with one possible percept after the action a is applied to the believe state b .

- Deterministic (local sensing)



(cont'd)

- Nondeterministic (local sensing + slippery world)



Solution with Partial Observation

Apply the AND-OR search algorithm.

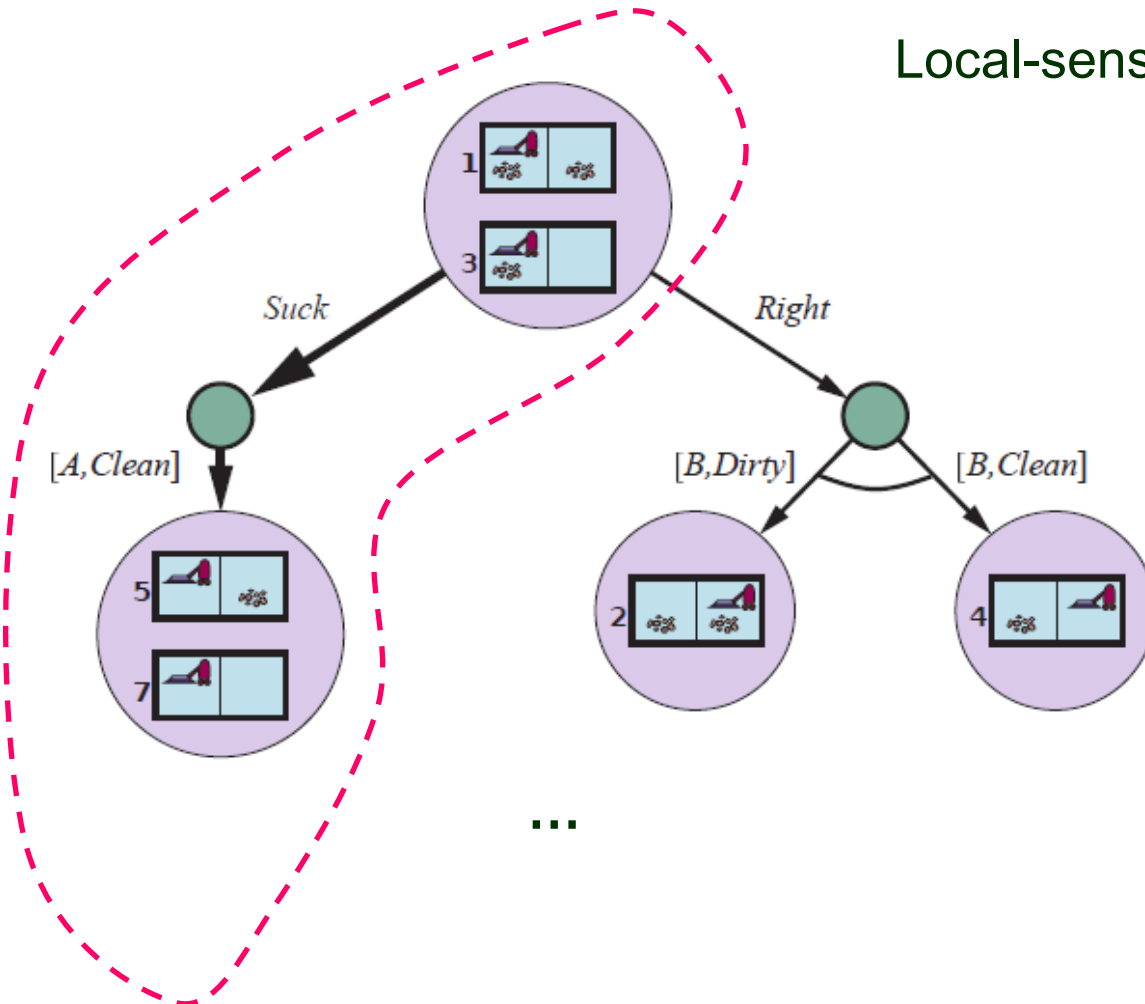
Local-sensing vacuum world

Initial percept: $[A, \text{Dirty}]$

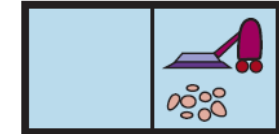
Solution:

```
Suck
Right
if b-state={6}
  then Suck
  else []
```

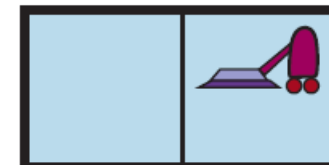
Goal:



6



8



Prediction-Update Cycles

- ♦ Test the condition and execute the appropriate branch.
- ♦ Maintain its belief state as it performs actions and receives percepts.

$$b' = \text{UPDATE}(\text{PREDICT}(b, a), o)$$

Prediction-Update Cycles

- ♦ Test the condition and execute the appropriate branch.
- ♦ Maintain its belief state as it performs actions and receives percepts.

$$b' = \text{UPDATE}(\text{PREDICT}(b, a), o)$$

Consider the local-sensing vacuum world in which

any square may become dirty *any time* unless being actively cleaned.

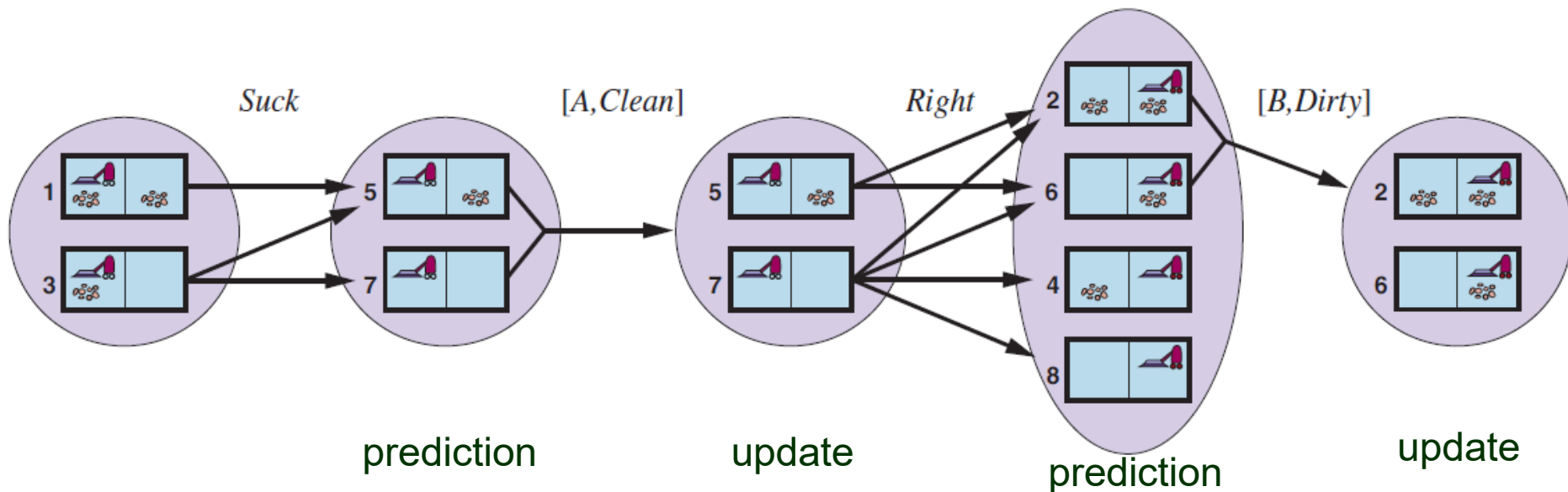
Prediction-Update Cycles

- ♦ Test the condition and execute the appropriate branch.
- ♦ Maintain its belief state as it performs actions and receives percepts.

$$b' = \text{UPDATE}(\text{PREDICT}(b, a), o)$$

Consider the local-sensing vacuum world in which

any square may become dirty *any time* unless being actively cleaned.



Robot Localization

Problem Use a map to determine where a robot is located with respect to its environment.

Robot Localization

Problem Use a map to determine where a robot is located with respect to its environment.

- Perfect sensing (by four sonar sensors)

Percept: 4 bits *NESW* (obstacle in the north, east, south, and west?)

Robot Localization

Problem Use a map to determine where a robot is located with respect to its environment.

- Perfect sensing (by four sonar sensors)

Percept: 4 bits *NESW* (obstacle in the north, east, south, and west?)

b: set of all locations

Robot Localization

Problem Use a map to determine where a robot is located with respect to its environment.

- Perfect sensing (by four sonar sensors)

Percept: 4 bits *NESW* (obstacle in the north, east, south, and west?)

b: set of all locations

↓ *NESW* = 1011 (obstacles in *N*, *S*, *W*)

$b_0 = \text{UPDATE}(b, 1011)$

Robot Localization

Problem Use a map to determine where a robot is located with respect to its environment.

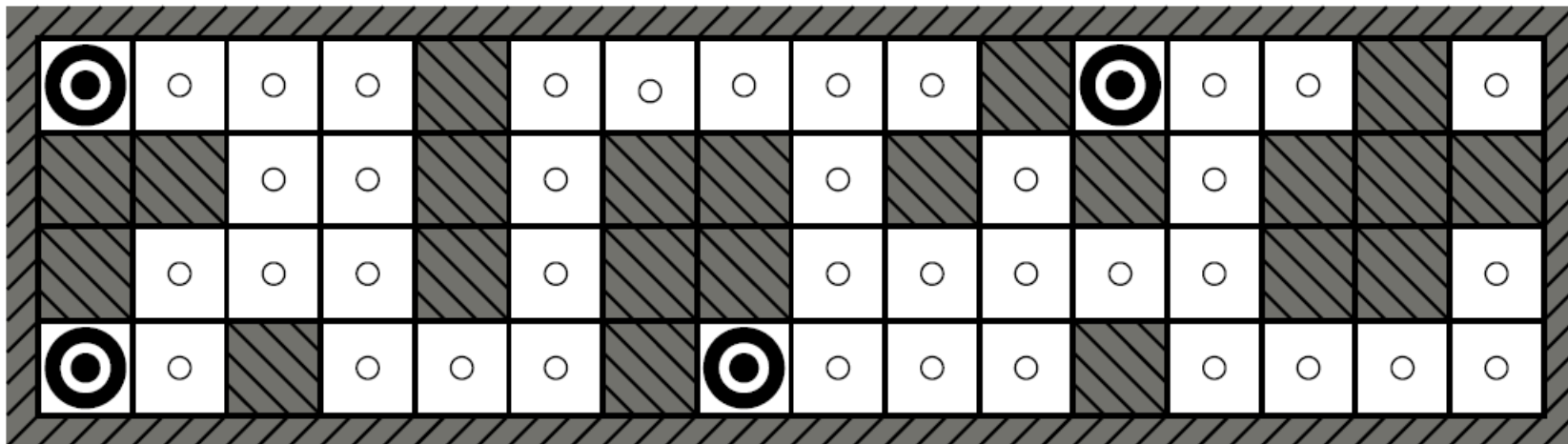
- Perfect sensing (by four sonar sensors)

Percept: 4 bits *NESW* (obstacle in the north, east, south, and west?)

b: set of all locations

↓ *NESW* = 1011 (obstacles in *N*, *S*, *W*)

*b*₀ = UPDATE(*b*, 1011)



Robot Localization

Problem Use a map to determine where a robot is located with respect to its environment.

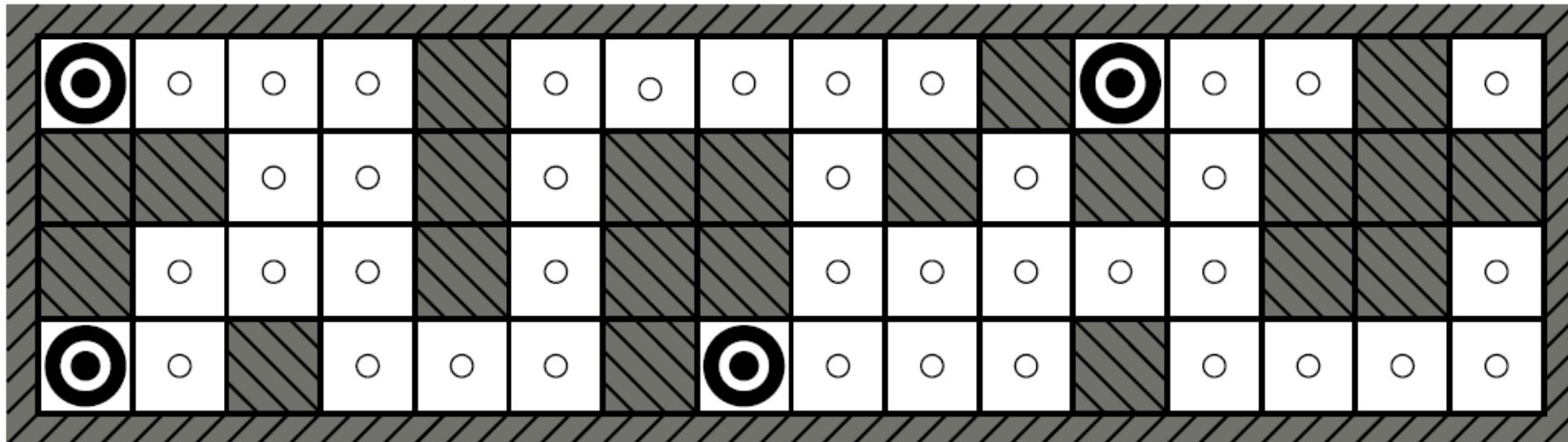
- Perfect sensing (by four sonar sensors)

Percept: 4 bits *NESW* (obstacle in the north, east, south, and west?)

b: set of all locations

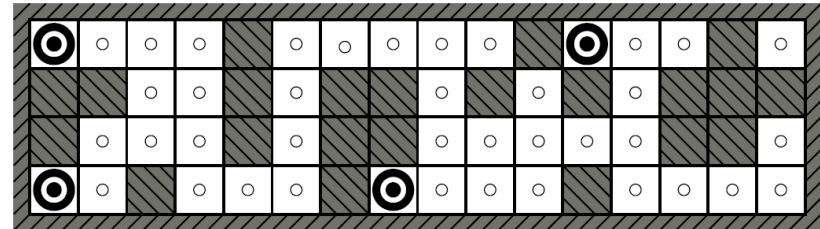
↓ *NESW* = 1011 (obstacles in *N*, *S*, *W*)

*b*₀ = UPDATE(*b*, 1011) // only four possible locations



Robot Localization

- **Nondeterministic navigation:** Action *Right* takes the robot randomly to one of the four adjacent squares.

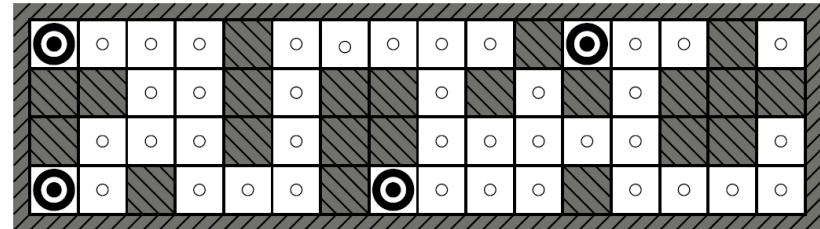


Robot Localization

- **Nondeterministic navigation:** Action *Right* takes the robot randomly to one of the four adjacent squares.

Next, the robot executes a *Right* action.

$$b_a = \text{PREDICT}(b_0, \textit{Right})$$



Robot Localization

- **Nondeterministic navigation:** Action *Right* takes the robot randomly to one of the four adjacent squares.

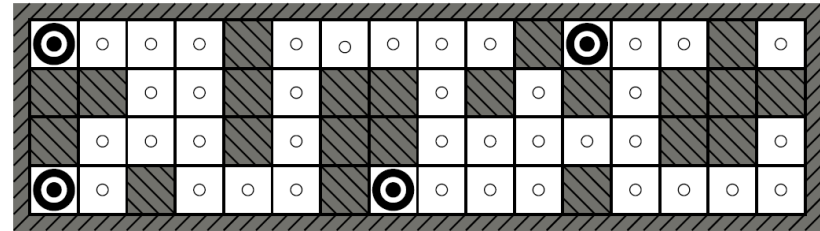
Next, the robot executes a *Right* action.

$$b_a = \text{PREDICT}(b_0, \text{Right})$$



$$NESW = 1010$$

$$b_1 = \text{Update}(b_a, 1010)$$



Robot Localization

- **Nondeterministic navigation:** Action *Right* takes the robot randomly to one of the four adjacent squares.

Next, the robot executes a *Right* action.

$$b_a = \text{PREDICT}(b_0, \text{Right})$$



$$\text{NESW} = 1010$$

$$b_1 = \text{Update}(b_a, 1010) \quad // \text{ One possible location!}$$

